public interface Set<E>

`public boolean contains(E e)`	Returns true if this set contains the specified element.
`public void add(E e)`	Adds the specified element to this set if it is not already present.
`public E remove(E e)`	Removes and returns the specified element from this set if it is present.
`public int size()`	Returns the number of elements in this set.

public interface Map<K, V>

`public boolean containsKey(K key)`	Returns true if this map contains a mapping for the specified key.
`public V get(K key)`	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
`public void put(K key, V value)`	Associates the specified value with the specified key in this map.
`public V remove(K key)`	Removes the mapping for a key from this map if it is present.
`public int size()`	Returns the number of key-value mappings in this map.

public interface List<E>

`public boolean contains(E element)`	Returns true if this list contains the specified element.
`public E get(int index)`	Returns the element at the specified position in this list.
`public void set(int index, E element)`	Replaces the element at the specified position in this list with the specified element.
`public void add(E e)`	Appends the specified element to the end of this list. Runs in Θ(1) time.
`public E remove(int index)`	Removes and returns the element at the specified position in this list.
`public int size()`	Returns the number of elements in this list.
`public List<E>(List<E> e)`	Constructor that takes in a List and creates a new List copy with the same elements.
`public List<E> subList(int a, int b)`	Returns a view of the portion of this list between index `a`, inclusive, and index `b`, exclusive.
`public boolean equals(Object o)`	Lists are defined to be equal if they contain the same elements in the same order.
`public static <E> List<E> of (E... elements)`	Returns an immutable list containing an arbitrary number of elements. Example: `List<Integer> lst = List.of(1, 2, 3);`

Implementations of Lists covered in class:

```
public class ArrayList<E> implements List<E> {...}
public class LinkedList<E> implements List<E> {...}
```

public class Random The provided Random methods all run in Θ(1) time.

`public Random()`	Creates a new random number generator (RNG).
`public Random(long seed)`	Creates a new RNG using a seed. If two instances of Random are created with the same seed, they will generate and return identical sequences of numbers.
`public int nextInt(int lower, int upper)`	Returns a pseudorandom int between `lower` (inclusive) and `upper` (exclusive), drawn from this RNG's sequence.

public class WeightedQuickUnion

`public WeightedQuickUnion(int n)`	Creates a WQU with n elements numbered 0 to n – 1 (inclusive).
`public boolean isConnected(int x, int y)`	Checks whether x and y belong to the same set.
`public void union(int x, int y)`	Unions the set that x belongs to and the set that y belongs to.

public class MinPQ<E extends Comparable<E>> extends PriorityQueue<E>

`public MinPQ()`	Creates a MinPQ<E> with elements ordered according to their natural ordering as defined by the compareTo method of the elements. Runs in Θ(1) time.
`public E getSmallest()`	Retrieves, but does not remove, the smallest element of this MinPQ, or returns null if this MinPQ is empty. Runs in Θ(1) time.
`public void insert(E e)`	Inserts the specified element into this MinPQ. Runs in O(log N) time.
`public E removeSmallest()`	Retrieves and removes the smallest element of this MinPQ, or returns null if this MinPQ is empty. The head of this heap is the element which is smallest according to the `compareTo` method. Runs in O(log N) time.
`public int size()`	Returns the size of the MinPQ. Runs in Θ(1) time.

public interface Deque<E>

`public E getFirst()`	Retrieves, but does not remove, the first element of this deque.
`public E getLast()`	Retrieves, but does not remove, the last element of this deque.
`public void addFirst(E e)`	Inserts the specified element at the front of this deque.
`public void addLast(E e)`	Inserts the specified element at the end of this deque.
`public E removeFirst()`	Retrieves and removes the first element of this deque.
`public E removeLast()`	Retrieves and removes the last element of this deque.
`public int size()`	Returns the number of elements in this deque.

Implementations of Deques covered in class:

```
public class ArrayDeque<E> implements Deque<E> {...}
public class LinkedListDeque<E> implements Deque<E> {...}
```

public class String

`public char charAt(int index)`	Returns the char value at the specified index.
`public int length()`	Returns the length of this string.
`public char[] toCharArray()`	Converts this string to a new character array.

public class Math The provided Math methods all run in Θ(1) time.

`public static double pow(double x, double y)`	Returns the value of x raised to the power of y.
`public static long floorMod(long x, long y)`	Returns the positive remainder when dividing x by y.
`public static double sqrt(double x)`	Returns the positive square root of x.
`public static int min(int x, int y)`	Returns the smaller of two int values.
`public static int max(int x, int y)`	Returns the greater of two int values.

`Integer.MIN_VALUE = -2147483648` = -2^{31} `Integer.MAX_VALUE = 2147483647` = $2^{31} - 1$